

Aircraft Animation using Euler Angles and Quaternions in OpenGL

Shreyansh Soni - 25377738

1 Introduction

This project implements an interactive aircraft animation system using OpenGL and GLM. The application allows real-time control of pitch, yaw, and roll using Euler angles and also supports quaternion-based rotation to overcome gimbal lock. Additional features include:

- On-screen UI using ImGui
- Gimbal lock demonstration
- Keyframed animation paths
- Quaternion interpolation using LERP and SLERP
- Motion easing (ease-in / ease-out)
- Figure-of-eight and circular trajectories

The goal is to demonstrate understanding of rotation mathematics and animation techniques.

2 Aircraft Model and Euler Rotation

The aircraft model is loaded using an OBJ file and transformed using a model matrix constructed from translation and Euler rotations.

Pitch, yaw, and roll are applied in Y-X-Z order:

```
modelMat = glm::translate(modelMat, position);

modelMat = glm::rotate(modelMat, glm::radians(yaw), glm::vec3(0,1,0));
modelMat = glm::rotate(modelMat, glm::radians(pitch), glm::vec3(1,0,0))
;
modelMat = glm::rotate(modelMat, glm::radians(roll), glm::vec3(0,0,1));
```

Keyboard controls:

- Arrow keys: Pitch and Yaw
- Q / E: Roll

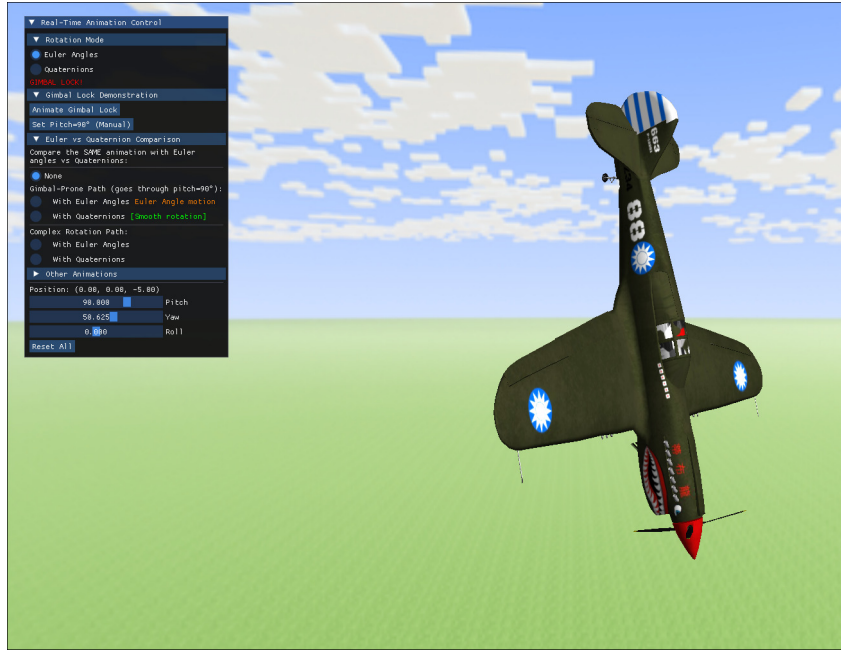
These values are displayed live through ImGui sliders.

3 Gimbal Lock Demonstration

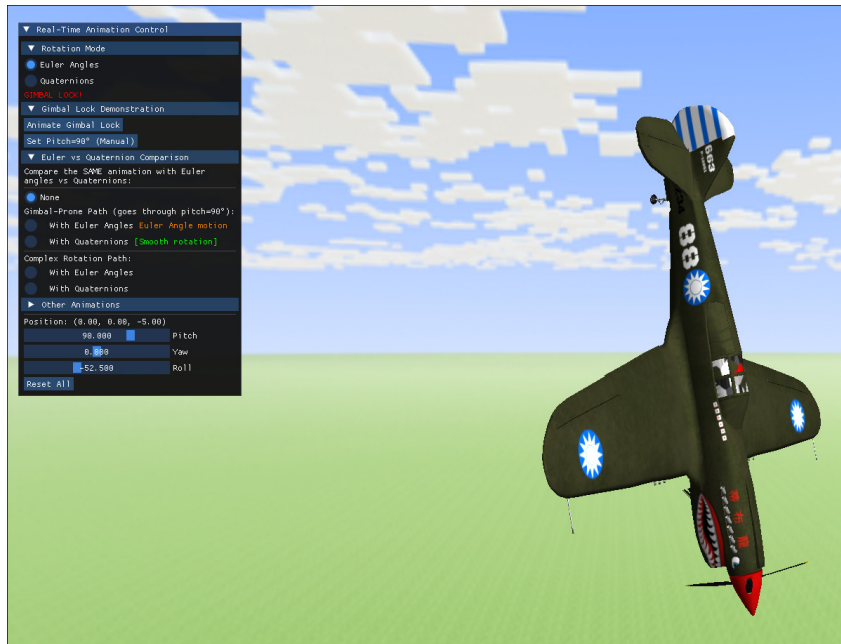
Gimbal lock occurs when pitch approaches $\pm 90^\circ$, causing loss of one rotational degree of freedom. This is detected in code:

```
bool isInGimbalLock() {  
    return (abs(pitch - 90.0f) < 10.0f ||  
            abs(pitch + 90.0f) < 10.0f);  
}
```

A dedicated animation automatically drives pitch to 90 degrees, after which yaw and roll become indistinguishable. The UI highlights this state with a warning.



(a) Yaw angle changed



(b) Roll angle changed

Figure 1: Gimbal lock demonstration: when pitch approaches 90° , yaw and roll rotations become aligned, producing identical aircraft motion despite changing different Euler angles.

4 Quaternion Rotation

To avoid gimbal lock, quaternion rotations are implemented.

Incremental rotations are applied via:

```
glm::quat pitchQuat =
glm::angleAxis(glm::radians(rotSpeed),
glm::vec3(1,0,0));
```

```
currentQuatRotation =  
pitchQuat * currentQuatRotation;
```

The quaternion is converted to a matrix using:

```
modelMat = modelMat * glm::mat4_cast(rotation);
```

This enables smooth, continuous rotation without axis collapse.

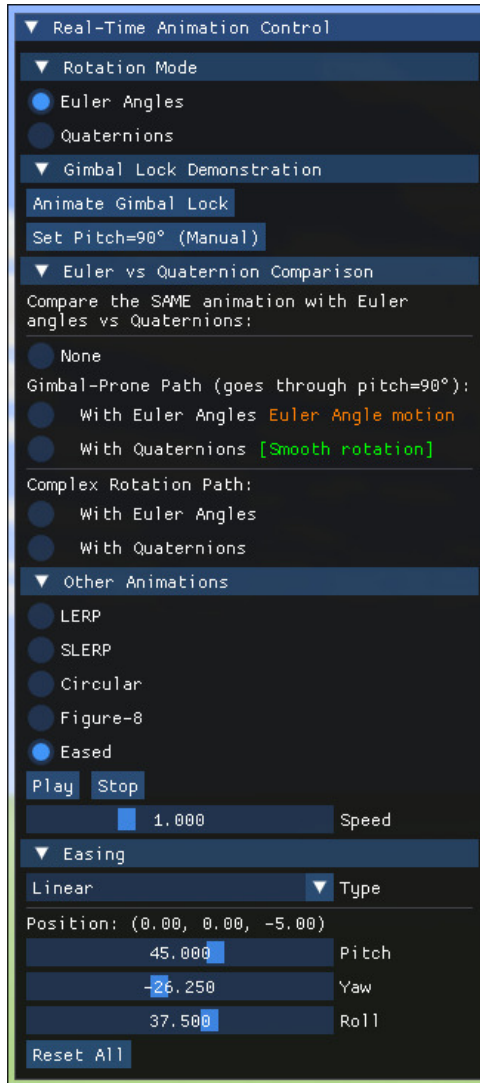
5 User Interface Feedback

ImGui provides real-time feedback:

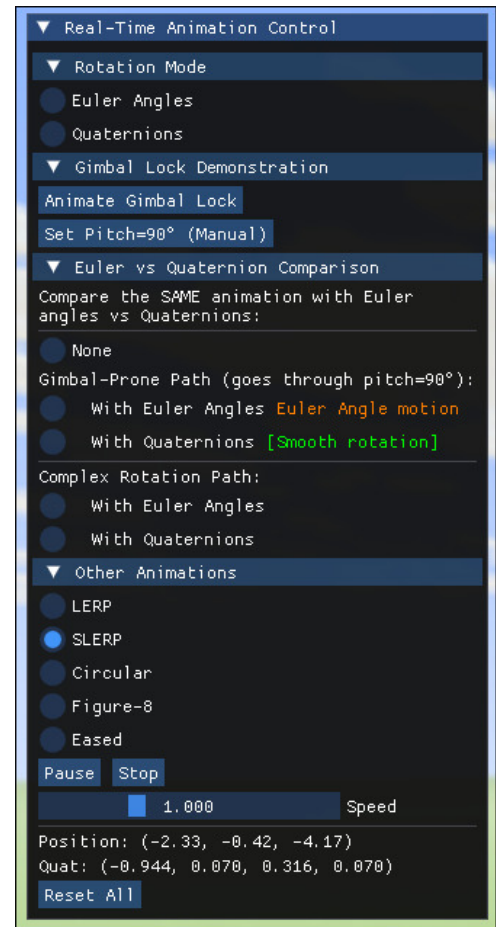
- Current position
- Euler angles or quaternion values
- Rotation mode selection
- Animation playback controls

The UI allows switching between:

- Euler mode
- Quaternion mode
- Gimbal lock demo
- Multiple animation paths



(a) Euler Angles UI



(b) Quaternions

Figure 2: The ImGui UI shows the necessary Log info for the object, and options to change between different modes

6 Keyframed Animation

The aircraft follows predefined keyframes consisting of position and orientation. Linear interpolation is used for position, while rotations use either:

- Euler interpolation
- Quaternion LERP
- Quaternion SLERP

Different paths include:

- Linear path
- Circular motion
- Figure-of-eight

- Gimbal-prone path

Playback speed and time are adjustable through sliders.

```
void getKeyframeTransform(float time, glm::vec3& outPos, glm::quat&
    outRot,
                        bool useSlerp, EasingType easing = EASE_LINEAR) {
if (quaternionKeyframes.empty()) return;

float totalTime = quaternionKeyframes.back().time;
time = fmod(time, totalTime);

for (size_t i = 0; i < quaternionKeyframes.size() - 1; i++) {
    if (time >= quaternionKeyframes[i].time && time <=
        quaternionKeyframes[i + 1].time) {
        float t = (time - quaternionKeyframes[i].time) /
            (quaternionKeyframes[i + 1].time - quaternionKeyframes
                [i].time);

        t = applyEasing(t, easing);

        outPos = lerpVec3(quaternionKeyframes[i].position,
            quaternionKeyframes[i + 1].position, t);

        if (useSlerp) {
            outRot = quatSlerp(quaternionKeyframes[i].rotation,
                quaternionKeyframes[i + 1].rotation, t);
        } else {
            outRot = quatLerp(quaternionKeyframes[i].rotation,
                quaternionKeyframes[i + 1].rotation, t);
        }
        return;
    }
}

outPos = quaternionKeyframes[0].position;
outRot = quaternionKeyframes[0].rotation;
}
```

7 Quaternion Interpolation

Two interpolation methods are implemented:

7.1 LERP

Linear interpolation between quaternions, faster but less accurate.

```
glm::quat quatLerp(const glm::quat& a, const glm::quat& b, float t) {
    glm::quat b_corrected = b;
    if (glm::dot(a, b) < 0.0f) {
        b_corrected = -b;
    }
    glm::quat result = a * (1.0f - t) + b_corrected * t;
    return glm::normalize(result);
}
}
```

7.2 SLERP

Spherical linear interpolation, preserving constant angular velocity.

These are selectable from the UI and clearly show smoother results compared to Euler animation.

```
glm::quat quatSlerp(const glm::quat& a, const glm::quat& b, float t) {
    glm::quat q1 = glm::normalize(a);
    glm::quat q2 = glm::normalize(b);

    float cosTheta = glm::dot(q1, q2);

    if (cosTheta < 0.0f) {
        q2 = -q2;
        cosTheta = -cosTheta;
    }

    if (cosTheta > 0.9995f) {
        return glm::normalize(q1 * (1.0f - t) + q2 * t);
    }

    float theta = acos(cosTheta);
    float sinTheta = sin(theta);

    float w1 = sin((1.0f - t) * theta) / sinTheta;
    float w2 = sin(t * theta) / sinTheta;

    return glm::normalize(q1 * w1 + q2 * w2);
}
```

8 Motion Smoothing

Easing functions are implemented:

- Linear
- Ease In Quad
- Ease Out Quad
- Ease In-Out Quad
- Cubic variants

These reduce abrupt motion by modifying the interpolation parameter.

```
float easeLinear(float t) { return t; }
float easeInQuad(float t) { return t * t; }
float easeOutQuad(float t) { return t * (2.0f - t); }
float easeInOutQuad(float t) { return t < 0.5f ? 2.0f * t * t : -1.0f +
    (4.0f - 2.0f * t) * t; }
float easeInCubic(float t) { return t * t * t; }
float easeOutCubic(float t) { return (--t) * t * t + 1.0f; }
float easeInOutCubic(float t) { return t < 0.5f ? 4.0f * t * t * t : (t
    - 1.0f) * (2.0f * t - 2.0f) * (2.0f * t - 2.0f) + 1.0f; }
```

9 Results

The system successfully demonstrates:

- Euler rotation with visible gimbal lock
- Quaternion-based rotation
- Smooth animation via SLERP
- Keyframed trajectories
- Motion easing
- Real-time UI feedback

Figure-of-eight and complex paths illustrate advanced animation control.



(a) Following figure 8 motion



(b) following Easin and Ease out motion

Figure 3: Showcasing different types of motions that can be achieved by keyframes

10 Conclusion

This project demonstrates practical implementation of aircraft animation using OpenGL. Euler angles highlight gimbal lock limitations, while quaternions provide robust rotational control. Keyframing, interpolation, and easing enhance realism. The comparison clearly shows why quaternions are preferred for 3D animation systems.